Python Inbuilt Data Structure

(3) set {}

Sets {} ---> (i) Define with curly brace (ii) Indexing & Slicing is not allowed (iii) set{} - if u pass only same data then order is allowed set -- Mix datatype might not get an order (iv)inbuit DT - .add() | .clear() | .copy() | .discard() | .remove()

In [1]:
```
s = {}      #sets define with curly brackets
s
```

Out[1]:
```
{}
```

In [2]:
```
type(s)     # {} --> Any value it will consider as dictionary
```

Out[2]:
```
dict
```

In [3]:
```
s = set()
s
```

Out[3]:
```
set()
```

In [4]:
```
type(s)    # If we add any value it will consider as set
```

Out[4]:
```
set
```

In [5]:
```
s1 ={10, 5, 45, 20, 40, 30, }
s1
```

Out[5]:
```
{5, 10, 20, 30, 40, 45}
```

In [6]:
```
type(s1)
```

Out[6]:
```
set
```

In [7]:
```
len(s1)
```

Out[7]:
```
6
```

In [8]:
```
s1
```

Out[8]:
```
{5, 10, 20, 30, 40, 45}
```

In [9]:
```
s1.add(90,67)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[9], line 1
----> 1 s1.add(90,67)

TypeError: set.add() takes exactly one argument (2 given)
```

In [10]:
```
s1.add(90)
```

In [11]:
```
s1
```

Out[11]:
```
{5, 10, 20, 30, 40, 45, 90}
```

In [12]:
```
s2 = s1.copy()
```

In [13]:
```
s2
```

Out[13]:
```
{5, 10, 20, 30, 40, 45, 90}
```

In [14]:
```
s2[3]    #Indexing is  not allowed in set
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[14], line 1
----> 1 s2[3]    #Indexing is  not allowed in set

TypeError: 'set' object is not subscriptable
```

In [15]:
```
s2[:]     # Slicing is not allowed in set
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[15], line 1
----> 1 s2[:]     # Slicing is not allowed in set

TypeError: 'set' object is not subscriptable
```

In [16]:
```
s3 = {'s', 2+4j, 4.6, False}
```

In [17]:
```
s3
```

Out[17]:
```
{(2+4j), 4.6, False, 's'}
```

In [18]:
```
print(s1)
print(s2)
print(s3)
```

```
{20, 5, 90, 40, 10, 45, 30}
{20, 5, 90, 40, 10, 45, 30}
{False, 's', 4.6, (2+4j)}
```

In [19]:
```
s1
```

Out[19]:
```
{5, 10, 20, 30, 40, 45, 90}
```

In [20]:
```
s1.pop()   #remove random value
```

Out[20]: 20

In [21]:
```
10 in s1
```

Out[21]: True

In [22]:
```
80 in s2
```

Out[22]: False

In [23]:
```
s1.discard(90)   # it will never through a error
```

In [24]:
```
s1
```

Out[24]: {5, 10, 30, 40, 45}

In [25]:
```
s1.remove(45)
```

In [26]:
```
s1
```

Out[26]: {5, 10, 30, 40}

SETS OPERATIONS

In [27]:
```
a1 = {1,2,3,4,5}
b1 = {4,5,6,7,8}
c1 = {8,9,10}
```

(i) UNION (|)

Combines elements from both sets.

In [28]:
```
a1.union(b1)
```

Out[28]: {1, 2, 3, 4, 5, 6, 7, 8}

In [29]:
```
a1.union(b1, c1)
```

Out[29]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

(ii) Intersection (&)

Common elements in both sets.

In [30]:
```
print(a1)
print(b1)
```

```
print(c1)
```

```
{1, 2, 3, 4, 5}
{4, 5, 6, 7, 8}
{8, 9, 10}
```

In [31]:
```
a1.intersection(b1)
```

Out[31]:
```
{4, 5}
```

In [32]:
```
a1.intersection(c1)
```

Out[32]:
```
set()
```

In [33]:
```
a1 & b1
```

Out[33]:
```
{4, 5}
```

In [34]:
```
b1 & c1
```

Out[34]:
```
{8}
```

In [35]:
```
a1 & b1 & c1
```

Out[35]:
```
set()
```

(iii) Difference (-)

Elements in first set but not in second.

In [36]:
```
print(a1)
print(b1)
print(c1)
```

```
{1, 2, 3, 4, 5}
{4, 5, 6, 7, 8}
{8, 9, 10}
```

In [37]:
```
a1.difference(b1)
```

Out[37]:
```
{1, 2, 3}
```

In [38]:
```
b1.difference(c1)
```

Out[38]:
```
{4, 5, 6, 7}
```

In [39]:
```
b1 - c1
```

Out[39]:
```
{4, 5, 6, 7}
```

In [40]:
```
a1 - b1
```

Out[40]:
```
{1, 2, 3}
```

(iv) Symmetric Difference (^)

Elements in either set but not both.

In [41]:
```
print(a1)
print(b1)
print(c1)
```

```
{1, 2, 3, 4, 5}
{4, 5, 6, 7, 8}
{8, 9, 10}
```

In [42]:
```
a1.symmetric_difference(b1)
```

Out[42]:
```
{1, 2, 3, 6, 7, 8}
```

In [43]:
```
b1.symmetric_difference(c1)
```

Out[43]:
```
{4, 5, 6, 7, 9, 10}
```

In [44]:
```
a1 ^ b1
```

Out[44]:
```
{1, 2, 3, 6, 7, 8}
```

In [45]:
```
b1 ^ c1
```

Out[45]:
```
{4, 5, 6, 7, 9, 10}
```

(v) difference_update()

It removes the common elements from the original set.

In [46]:
```
print(a1)
print(b1)
print(c1)
```

```
{1, 2, 3, 4, 5}
{4, 5, 6, 7, 8}
{8, 9, 10}
```

In [47]:
```
a1.difference_update(b1)
```

In [48]:
```
a1
```

Out[48]:
```
{1, 2, 3}
```

In [49]:
```
b1.difference_update(c1)
```

In [50]:
```
b1
```

Out[50]:
```
{4, 5, 6, 7}
```

(vi) Superset (>=)

Checks if first set is superset of second set.

In [51]:
```
print(a1)
print(b1)
print(c1)
```

```
{1, 2, 3}
{4, 5, 6, 7}
{8, 9, 10}
```

In [52]:
```
s5  = {1,2,3,4,5,6,7,8,9}
s6 = {3,4,5,6,7,8}
s7 = {10,20,30,40}
```

In [53]:
```
s5.issuperset(s6)
```

Out[53]:
```
True
```

In [54]:
```
s5 >= s6
```

Out[54]:
```
True
```

In [55]:
```
s6.issuperset(s5)
```

Out[55]:
```
False
```

In [56]:
```
s6 >= s7
```

Out[56]:
```
False
```

(vii) Subset (<=)

In [ ]:
```

```

In [57]:
```
s5  = {1,2,3,4,5,6,7,8,9}
s6 = {3,4,5,6,7,8}
s7 = {10,20,30,40}
```

In [58]:
```
s5.issubset(s6)
```

Out[58]:
```
False
```

In [59]:
```
s6.issubset(s5)
```

Out[59]:
```
True
```

In [60]:
```
s6 <= s7
```

Out[60]: False

In [61]:
```
s6 <= s5
```

Out[61]: True

(viii) Disjoint

First set and second set are disjoint if they have no common elements.

In [62]:
```
s5.isdisjoint(s6)
```

Out[62]: False

In [63]:
```
s7.isdisjoint(s5)
```

Out[63]: True

In [64]:
```
s6.isdisjoint(s7)
```

Out[64]: True

In [ ]:

RANGE

#range() is used to generate a sequence of numbers.

In [65]:
```
range()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[65], line 1
----> 1 range()

TypeError: range expected at least 1 argument, got 0
```

In [66]:
```
range(5)
```

Out[66]: range(0, 5)

In [67]:
```
list(range(5))
```

Out[67]: [0, 1, 2, 3, 4]

In [68]:
```
list(range(1,5))
```

Out[68]: [1, 2, 3, 4]

In [69]:
```
list(range(1, 10, 3))
```

Out[69]: [1, 4, 7]

In [70]:
```
range(10)
```

Out[70]: range(0, 10)

In [71]:
```
r = range(10)
r
```

Out[71]: range(0, 10)

In [72]:
```
for  i in r:
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
```

In [73]:
```
range(1, 10, 3, 4)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[73], line 1
----> 1 range(1, 10, 3, 4)

TypeError: range expected at most 3 arguments, got 4
```

DICTIONARY

A dictionary is a collection of key-value pairs where each key is unique and used to access its value. || Written using { }

In [74]:
```
d = {}
type(d)
```

Out[74]: dict

In [75]:
```
d1 = {'l' :10, 'm' :2.3, "n" :False, 'o' :4+2j}
d1
```

Out[75]: {'l': 10, 'm': 2.3, 'n': False, 'o': (4+2j)}

In [76]:
```
d1.keys()
```

Out[76]: dict_keys(['l', 'm', 'n', 'o'])

In [77]:
```
d1.values()
```

Out[77]: dict_values([10, 2.3, False, (4+2j)])

In [78]:
```
d1.items()
```

Out[78]: dict_items([('l', 10), ('m', 2.3), ('n', False), ('o', (4+2j))])

In [79]:
```
d2 = d1.copy()
```

In [80]:
```
d2
```

Out[80]: {'l': 10, 'm': 2.3, 'n': False, 'o': (4+2j)}

In [81]:
```
d1 == d2
```

Out[81]: True

In [82]:
```
print(d1)
print(d2)
```

```
{'l': 10, 'm': 2.3, 'n': False, 'o': (4+2j)}
{'l': 10, 'm': 2.3, 'n': False, 'o': (4+2j)}
```

In [83]:
```
d1
```

Out[83]: {'l': 10, 'm': 2.3, 'n': False, 'o': (4+2j)}

In [84]:
```
d1['n']
```

Out[84]: False

In [85]:
```
d1[10]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[85], line 1
----> 1 d1[10]

KeyError: 10
```

In [86]:
```
mydict3 = {1:'one', 2:'two', 'A':{'Name':'Afjal', 'Age':23}, 'B':('Bat', 'cat', 'ha
mydict3
```

Out[86]:
```
{1: 'one',
 2: 'two',
 'A': {'Name': 'Afjal', 'Age': 23},
 'B': ('Bat', 'cat', 'hat')}
```

In [87]:
```
mydict3
```

Out[87]:
```
{1: 'one',
 2: 'two',
 'A': {'Name': 'Afjal', 'Age': 23},
 'B': ('Bat', 'cat', 'hat')}
```

In [88]:
```
keys = {'a', 'b', 'c', 'd'}
mydict3 = dict.fromkeys(keys)   # create a dict from a sequences of keys
mydict3
```

Out[88]:
```
{'a': None, 'd': None, 'b': None, 'c': None}
```

In [89]:
```
keys = {'a', 'b', 'c', 'd'}
value = 20
mydict3 = dict.fromkeys(keys, value)   # create a dict from a sequences of keys
mydict3
```

Out[89]:
```
{'a': 20, 'd': 20, 'b': 20, 'c': 20}
```

In [90]:
```
keys = {'a', 'b', 'c', 'd'}
value = [10,20,30]

mydict3 = dict.fromkeys(keys, value)   # create a dict from a sequences of keys
mydict3
```

Out[90]:
```
{'a': [10, 20, 30], 'd': [10, 20, 30], 'b': [10, 20, 30], 'c': [10, 20, 30]}
```

In [91]:
```
value.append(40)
mydict3
```

Out[91]:
```
{'a': [10, 20, 30, 40],
 'd': [10, 20, 30, 40],
 'b': [10, 20, 30, 40],
 'c': [10, 20, 30, 40]}
```

In [92]:
```
d1.get('n')
```

Out[92]:
```
False
```

In [93]:
```
mydict3
```

Out[93]:
```
{'a': [10, 20, 30, 40],
 'd': [10, 20, 30, 40],
 'b': [10, 20, 30, 40],
 'c': [10, 20, 30, 40]}
```

In [94]:
```
mydict3.get('d')
```

Out[94]:
```
[10, 20, 30, 40]
```

In [95]:
```
d1
```

Out[95]:
```
{'l': 10, 'm': 2.3, 'n': False, 'o': (4+2j)}
```

In [96]:
```
d1[10] = 'ten'
```

In [97]:
```
d1
```

Out[97]:
```
{'l': 10, 'm': 2.3, 'n': False, 'o': (4+2j), 10: 'ten'}
```

In [98]:
```
d1.pop()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[98], line 1
----> 1 d1.pop()

TypeError: pop expected at least 1 argument, got 0
```

In [99]:
```
d1.pop(5)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[99], line 1
----> 1 d1.pop(5)

KeyError: 5
```

In [100…
```
d1.pop('n')
```

Out[100…
```
False
```

In [101…
```
d1
```

Out[101…
```
{'l': 10, 'm': 2.3, 'o': (4+2j), 10: 'ten'}
```

In [102…
```
d1.popitem()
```

Out[102…
```
(10, 'ten')
```

In [103…
```
d1
```

Out[103…
```
{'l': 10, 'm': 2.3, 'o': (4+2j)}
```

In [104…
```
all(d1)
```

Out[104…
```
True
```

In [105…
```
any(d1)
```

Out[105…
```
True
```

In [106…
```
d1[0] =0
```

In [107…
```
d1
```

Out[107…
```
{'l': 10, 'm': 2.3, 'o': (4+2j), 0: 0}
```

In [108…
```
all(d1)
```

Out[108…
```
False
```

In [109…
```
any(d1)
```

Out[109… True

In [110…
```
d1
```

Out[110… {'l': 10, 'm': 2.3, 'o': (4+2j), 0: 0}

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: